**UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO**

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE INGENIERÍA DE SOFTWARE

# Ingeniería de Requisitos

## Sílabo y material de estudio

Código de la asignatura: **ISR-401**

Cuarto nivel — 3 créditos — 96 horas presenciales

*Material de estudio elaborado conforme al estándar internacional **IEEE/ISO/IEC 29148:2018**, la guía **SWEBOK v4.0 (2024)**, los modelos de calidad **ISO/IEC 25010:2023** y la segunda edición del manual de Pohl (Springer, 2024).*

**Período académico: 2025 – 2026**

Quevedo – Los Ríos – Ecuador

*«La especificación de requisitos es el momento de la ingeniería del software en el que se decide, en última instancia, si el sistema va a resolver el problema correcto. Todo lo que viene después es construcción; lo que viene antes es ingeniería de requisitos. »*

— *adaptado de Klaus Pohl [**Pohl2024**]*

---

**Aviso:** Este material acompaña al *Programa Analítico* oficial de la asignatura aprobado por la Facultad de Ciencias de la Computación. Los contenidos están enmarcados en los estándares y guías internacionales que se citan a lo largo del texto y se sustentan en bibliografía verificable, según las normas IEEE.

# Índice general

# Índice de figuras

# Índice de tablas

Semana 1

# Técnicas avanzadas de elicitación y modelado de metas

*Un prototipo vale más que mil palabras de requisito: muestra lo que el analista ha entendido y le permite al stakeholder señalar exactamente lo que está mal. La representación visual convierte las suposiciones en afirmaciones verificables.*
adaptado de **Wiegers2013**

**Objetivos de aprendizaje de la semana**

Al finalizar esta cuarta semana, el estudiante estará en capacidad de:

1. Aplicar el **prototipado** —de baja y alta fidelidad— como técnica activa de elicitación y validación temprana [**Pohl2024**, **Wiegers2013**, **Floyd1984**].
2. Construir **modelos i*** (i-estrella) de actores y dependencias para analizar el contexto organizacional y derivar metas de sistema [**Yu2011**, **Pohl2024**, **IREB_CPRE_2024**].
3. Construir **grafos de metas KAOS** para descomponer objetivos de negocio en requisitos operacionalizables [**vanLamsweerde2001**, **Pohl2024**].
4. Modelar procesos de negocio con **Business Process Model and Notation (BPMN) 2.0** e identificar los eventos de negocio que generan requisitos funcionales [**Robertson2012**, **OMG_BPMN_2013**].
5. Redactar **historias de usuario** completas con criterios de aceptación en formato Acceptance Test-Driven Development (ATDD)/Gherkin para proyectos ágiles [**Cohn2004**, **Adzic2011**, **North2006**].
6. Seleccionar y combinar técnicas avanzadas de elicitación según el tipo de sistema y contexto del proyecto [**Pohl2024**, **SWEBOK_v4_2024**].

## 1.1 Tema 6. Técnicas avanzadas de elicitación

### 1.1.1 Subtema 6.1. Prototipado como técnica de elicitación

#### El prototipo como herramienta de comunicación

**Definición**

**Prototipo (en IR).** Representación parcial, simplificada y construida rápidamente del sistema o de alguno de sus aspectos, cuyo propósito no es ser el sistema final sino *hacer visibles las suposiciones del analista* y provocar retroalimentación temprana y concreta de los stakeholders [**Pohl2024**, **Wiegers2013**, **Floyd1984**].

Floyd [**Floyd1984**] fue quien formalizó el concepto de prototipado como técnica de explo-

ración de requisitos, diferenciándolo del prototipado como técnica de diseño. La distinción es crucial: en Ingeniería de Requisitos (IR), el prototipo se construye para *aprender sobre los requisitos*, no para convertirse en el sistema. Esta orientación tiene una consecuencia práctica directa: un prototipo de elicitación puede —y debe— ser descartado en cuanto cumple su función [**Pohl2024**].

### Taxonomía de prototipos

**Tabla 1.1.** *Taxonomía de prototipos en IR por fidelidad y técnica. Adaptado de [***Pohl2024***, **Wiegers2013***].*

| Tipo | Descripción | Costo/tiempo | Cuándo usar |
|---|---|---|---|
| *Paper mock-up* (baja fidelidad) | Dibujos en papel de pantallas o flujos. | Horas; cero costo digital. | Exploración inicial; validar estructura de navegación. |
| *Wireframe* digital (baja-media) | Esquemas en Figma, Balsamiq, draw.io, sin colores ni estilos reales. | 1–3 días. | Validar flujo y arquitectura de información. |
| *Mock-up* (alta fidelidad) | Diseño visual completo pero no funcional. | 3–7 días. | Validar apariencia, usabilidad (pruebas A/B). |
| Prototipo funcional (evolutivo) | Código real, funcionalidad parcial; puede evolucionar. | Semanas. | Sistemas con alta incertidumbre de dominio. |
| *Spike* / POC | Experimento técnico muy limitado para verificar factibilidad. | 1–3 días. | Validar que una integración o tecnología es posible. |

### Prototipado desechable vs. evolutivo

La literatura [**Pohl2024**, **Wiegers2013**, **Floyd1984**] distingue dos estrategias opuestas cuya confusión genera problemas frecuentes en proyectos:

**Prototipado desechable (*throwaway*).** El prototipo se construye con el único propósito de elicitar o validar requisitos. Una vez que los requisitos quedan claros, el prototipo se descarta. Ventaja: no hay presión de calidad de código. Riesgo: el cliente puede pedir «quedar con el prototipo», lo que produce un sistema frágil construido sobre una base exploratoria.

**Prototipado evolutivo.** El prototipo se convierte progresivamente en el sistema final. Ventaja: elimina el desperdicio de reescribir. Riesgo: acumula deuda técnica si no se refactoriza activamente. Requiere prácticas de ingeniería rigurosas desde el inicio.

**Advertencia / Error frecuente**

**Error frecuente.** El cliente ve un prototipo de alta fidelidad y asume que el sistema está «casi listo». Un wireframe interactivo de dos días de trabajo puede verse como una semana de desarrollo para alguien sin experiencia en software. Gestionar las expectativas es tan importante como construir el prototipo [**Wiegers2013**].

**Ejemplo en Ecuador**

**Caso: sistema de gestión de becas Secretaría Nacional de Educación Superior, Ciencia, Tecnología e Innovación (SENESCYT).** El equipo de análisis construyó en tres días un *paper mock-up* del flujo de postulación a becas. Al presentarlo a los directores de la unidad beneficiaria, descubrieron que el proceso real tenía una etapa de «preselección técnica»que ningún documento mencionaba y que era gestionada informalmente por los técnicos mediante correo electrónico. Esa etapa se convirtió en cinco nuevos requisitos funcionales del sistema.

### 1.1.2 Subtema 6.2. Modelado orientado a metas: i* y KAOS

#### ¿Por qué modelar metas?

El modelado orientado a metas (*goal-oriented requirements engineering*) aborda una limitación fundamental de la IR clásica: los requisitos funcionales describen *qué hace el sistema*, pero no responden a *por qué ese sistema existe* ni *para quién*. Esta falta de conexión entre los requisitos y las metas organizacionales es una de las causas más frecuentes de sistemas técnicamente correctos pero estratégicamente inútiles [**vanLamsweerde2001**, **Yu2011**, **Pohl2024**].

**Definición**

**Modelado orientado a metas.** Paradigma de IR que centra el proceso en la identificación, representación y refinamiento de los *objetivos* (metas) de los stakeholders y de la organización, para derivar de ellos los requisitos del sistema de forma trazable [**vanLamsweerde2001**, **Pohl2024**].

Existen dos frameworks consolidados con comunidades de práctica activas y soporte en herramientas: el **framework i*** (i-estrella) de Eric Yu [**Yu2011**] y el framework **KAOS** de van Lamsweerde [**vanLamsweerde2001**]. Ambos se incluyen en el corpus del IREB CPRE v3.2.0 y en SWEBOK v4 [**SWEBOK_v4_2024**].

#### El framework i* (i-estrella)

**Definición**

**Framework i*** (i-estrella). Marco de modelado orientado a actores sociales, propuesto por Eric Yu [**Yu2011**], que concibe el sistema como un conjunto de **actores** (agentes, roles, posiciones) que mantienen **dependencias** entre sí: de recursos, de tareas, de metas y de *softgoals* (metas blandas).

El framework i* se usa en la IR con dos modelos complementarios [**Yu2011**, **Pohl2024**]:

- **Modelo SD** (*Strategic Dependency*): muestra las dependencias entre actores externos al

sistema, sin entrar en sus intenciones internas. Responde a la pregunta: *¿quién depende de quién, y en qué?*

- **Modelo SR** (*Strategic Rationale*): abre la «frontera»de cada actor y muestra sus metas, tareas, recursos y *softgoals* internos, y cómo las contribuciones de otros actores satisfacen (o inhiben) esas metas.

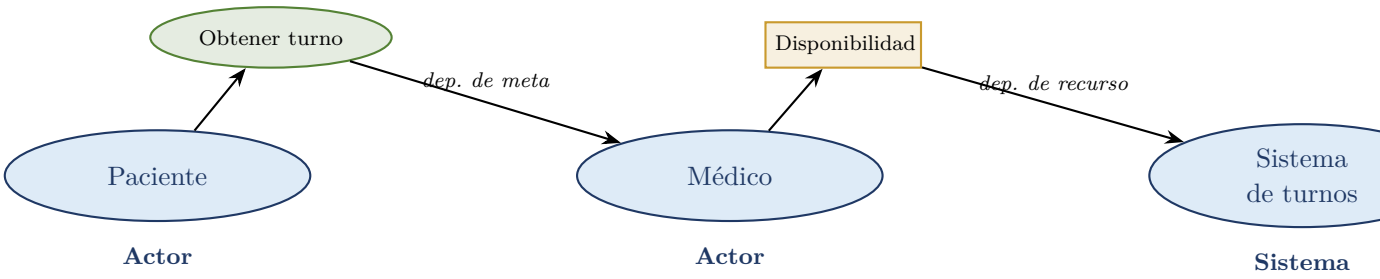



**Figura 1.1.** *Modelo SD simplificado en i* para el sistema de turnos del Ministerio de Salud Pública del Ecuador (MSP). Las dependencias muestran lo que cada actor necesita de los demás. Adaptado de [***Yu2011***, ***Pohl2024****].*

**Notación básica de i*.** Los cuatro tipos de dependencias del modelo SD son [**Yu2011**]:

- **Dependencia de meta** (círculo): el *depender* depende del *dependee* para que se alcance una meta.
- **Dependencia de tarea** (hexágono): el *depender* necesita que el *dependee* ejecute una tarea específica.
- **Dependencia de recurso** (rectángulo): se necesita una entidad física o de información.
- **Dependencia de softgoal** (nube): se necesita que se satisfaga un atributo de calidad (NFR).

### El framework KAOS

**Definición**

**KAOS** (*Keep All Objectives Satisfied*). Framework de modelado orientado a metas propuesto por van Lamsweerde y la Université catholique de Louvain. Organiza los requisitos en una *jerarquía de metas* que se descompone hasta llegar a metas operacionalizables (requisitos concretos) o expectativas sobre el entorno (suposiciones) [**vanLamsweerde2001**, **Pohl2024**].

La jerarquía KAOS utiliza cuatro operadores de descomposición [**vanLamsweerde2001**]:

**Tabla 1.2.** *Operadores de descomposición de metas en KAOS* [**vanLamsweerde2001**].

| Operador | Significado | Ejemplo |
|---|---|---|
| AND-refinement | La meta padre se cumple solo si *todas* las sub-metas se cumplen. | «Gestionar turnos»= «Reservar turno»AND «Cancelar turno»AND «Reprogramar turno». |
| OR-refinement | La meta padre se cumple si *alguna* sub-meta se cumple (alternativas). | «Autenticar usuario»= «Contraseña»OR «Biometría»OR «Token OTP». |
| Operacionalización | La sub-meta ya es un requisito atómico asignable al sistema. | «El sistema deberá enviar confirmación por SMS al confirmar la reserva.» |
| Suposición (assumption) | Condición del entorno que el sistema asume verdadera. | «El paciente tiene número de celular activo en el registro.» |

**Ejemplo en Ecuador**

**Aplicación KAOS: sistema de turnos MSP de Quevedo.** Meta de negocio de alto nivel: «Reducir la espera del paciente a menos de 30 minutos en el 90 % de los casos»(OR-refinement):

- Alternativa A: Gestión eficiente de turnos (AND): reserva en línea + recordatorio automático + gestión de no asistencia.
- Alternativa B: Redistribución de carga entre médicos (AND): monitoreo de tiempos + reasignación dinámica.

El equipo eligió la Alternativa A porque la Alternativa B requería cambios organizacionales fuera del alcance del proyecto de software. Esta decisión queda trazada en el grafo KAOS y justifica el alcance del ERS.

### 1.1.3 Subtema 6.3. Modelado de procesos de negocio con BPMN

#### BPMN como fuente de requisitos funcionales

**Definición**

**BPMN 2.0.** Notación estándar de la Object Management Group (OMG) para modelar procesos de negocio mediante diagramas de flujo enriquecidos con semántica de colaboración y coreografía. En IR, los diagramas BPMN sirven para capturar el proceso *AS-IS*, identificar los **eventos de negocio** que disparan acciones del sistema y derivar de ellos los requisitos funcionales correspondientes [**Robertson2012**, **OMG_BPMN_2013**, **Pohl2024**].

Robertson & Robertson [**Robertson2012**] proponen el concepto de **evento de negocio** como punto de partida de la elicitación: cada cosa que ocurre en el mundo del cliente y que requiere una respuesta del sistema es un evento de negocio. Identificar estos eventos de forma exhaustiva garantiza que no se olvide ninguna funcionalidad principal.

### Elementos BPMN esenciales para IR

**Tabla 1.3.** *Elementos BPMN 2.0 más relevantes para la elicitación de requisitos [**OMG_BPMN_2013**].*

| Elemento | Descripción | Requisito derivado |
|---|---|---|
| Evento de inicio | Algo que dispara el proceso. | RF de inicio de flujo. |
| Tarea de usuario | Trabajo manual realizado por una persona. | RF de interfaz de usuario. |
| Tarea de servicio | Acción automática del sistema. | RF de procesamiento o integración. |
| Gateway exclusivo (XOR) | El flujo toma un camino u otro, no ambos. | Lógica condicional en RF; casos alternativos. |
| Gateway paralelo (AND) | Los flujos se ejecutan en paralelo. | RF de procesamiento concurrente. |
| Evento intermedio | Algo que ocurre en mitad del proceso. | RF de notificación, espera o excepción. |
| Lane (carril) | Responsabilidad de un actor o sistema. | Asignación de RF al actor correcto. |

**Técnica de derivación de requisitos desde BPMN.** El procedimiento para convertir un diagrama BPMN en candidatos a requisitos funcionales es el siguiente [**Robertson2012**, **Pohl2024**]:

1. Por cada **tarea de usuario**, se genera un RF de tipo «el sistema deberá permitir al actor X realizar Y».
2. Por cada **tarea de servicio**, se genera un RF de tipo «el sistema deberá ejecutar automáticamente Y».
3. Por cada **gateway XOR**, se generan RF para cada camino, más un RF de evaluación de la condición.
4. Por cada **evento intermedio de tipo mensaje o timer**, se genera un RF de notificación o temporización.
5. Por cada **lane**, se revisa si el actor tiene acceso adecuado: RF de seguridad / autorización.

**Ejemplo en Ecuador**

**Proceso de facturación electrónica Servicio de Rentas Internas (Ecuador) (SRI).** El proceso BPMN tiene: evento inicio (el contador emite una factura), tarea de usuario (ingresa datos del receptor), tarea de servicio (sistema genera XML firmado), tarea de servicio (sistema envía al SRI), gateway XOR (¿respuesta de autorización?), evento final (factura autorizada / factura rechazada). De este único proceso BPMN se derivan al menos 8 requisitos funcionales y 3 no funcionales (tiempo de respuesta del SRI, disponibilidad del certificado digital, reintentos ante fallo de red) [**SRI_FE_2024**].

### 1.1.4 Subtema 6.4. Historias de usuario y técnicas ágiles de elicitación

#### Historias de usuario

**Definición**

**Historia de usuario.** Formato breve de descripción de un requisito, propio del desarrollo ágil, con la estructura: *«Como [rol], quiero [acción/capacidad], para [beneficio/valor]»*. Complementada con **criterios de aceptación** que permiten saber cuándo la historia está *terminada* (done) [**Cohn2004**, **Schwaber2020**].

Cohn [**Cohn2004**] introduce el acrónimo **INVEST** como criterio de calidad de una historia de usuario:

**Tabla 1.4.** *Criterios INVEST para evaluar la calidad de una historia de usuario [**Cohn2004**].*

| Criterio | Significado |
|---|---|
| **I**ndependent | La historia puede desarrollarse y entregarse de forma independiente. |
| **N**egotiable | Su contenido se negocia entre el equipo y el Product Owner hasta que se desarrolle. |
| **V**aluable | Aporta valor real al usuario o al negocio; si no aporta valor, no es una historia. |
| **E**stimable | El equipo puede estimar el esfuerzo necesario para implementarla. |
| **S**mall | Cabe en un sprint (habitualmente menos de 8 puntos de historia). |
| **T**estable | Tiene criterios de aceptación que permiten verificar si está hecha correctamente. |

#### Criterios de aceptación en formato Gherkin (ATDD)

El **ATDD** (Acceptance Test-Driven Development) propone escribir los criterios de aceptación de una historia antes de implementarla, en un lenguaje comprensible tanto para técnicos como para no técnicos. La notación **Gherkin** (usada por Cucumber y Behave) sigue la estructura **Given / When / Then** [**Adzic2011**, **North2006**]:

**Estructura Gherkin (Given/When/Then):**

**Scenario**: [nombre del escenario]
**Given** [contexto / precondición]
**When** [acción del usuario o del sistema]
**Then** [resultado esperado / poscondición]
**And** [condición adicional opcional]

**Ejemplo en Ecuador**

**Historia de usuario y criterios de aceptación: matrícula Universidad Técnica Estatal de Quevedo (UTEQ).**
*Historia:* «Como estudiante de cuarto nivel de Ingeniería de Software, quiero matricularme en línea en mis asignaturas del semestre, para evitar desplazarme a las instalaciones de la universidad.»
*Criterio de aceptación 1:*
**Given** el estudiante ha iniciado sesión con su cédula y contraseña
**When** selecciona la asignatura «Ingeniería de Requisitos»
**Then** el sistema valida que cumple el prerrequisito (Fundamentos de IS)
**And** registra la matrícula y genera el comprobante en PDF.
*Criterio de aceptación 2 (flujo alternativo):*
**Given** el estudiante ha iniciado sesión
**When** intenta matricularse en «Diseño de Software»sin haber aprobado el prerrequisito
**Then** el sistema muestra el mensaje: «No cumple el prerrequisito: Fundamentos de IS (reprobada en período 2024-I)»
**And** no registra la matrícula.

### Épicas, temas e historias: la jerarquía ágil

**Tabla 1.5.** *Jerarquía de artefactos de requisitos en metodologías ágiles. Adaptado de [***Cohn2004***, **Schwaber2020***].*

| Nivel | Descripción | Ejemplo (sistema de matrículas) |
|---|---|---|
| Tema (Theme) | Área de capacidad de alto nivel. | Matrícula académica. |
| Épica (Epic) | Historia muy grande; debe desglosarse en sub-historias. | «Como estudiante, quiero gestionar mi matrícula completa.» |
| Historia de usuario | Unidad entregable en un sprint. | «Como estudiante, quiero matricularme en línea.» |
| Tarea (Task) | Unidad de trabajo técnico dentro de la historia. | Crear endpoint REST /api/matricula, diseñar UI del formulario. |

### Técnicas complementarias ágiles de elicitación

Más allá de las historias de usuario, el ecosistema ágil dispone de técnicas específicas de elicitación que complementan al Product Owner [**Cohn2004**, **Adzic2011**, **IREB_CPRE_2024**]:

- **Story mapping**: técnica visual de Jeff Patton que organiza las historias en un mapa bidimensional (flujo de usuario horizontal × nivel de detalle vertical), permitiendo identificar el Minimum Viable Product (Producto Mínimo Viable) (MVP) y planificar releases.
- **Event storming**: taller de exploración del dominio propuesto por Alberto Brandolini, basado en post-its de colores que representan eventos de dominio, comandos, agregados y políticas. Muy efectivo para descubrir el modelo de dominio y los eventos de negocio.
- **Impact mapping**: técnica de Gojko Adzic [**Adzic2011**] que vincula las historias con los

objetivos de negocio a través de actores e impactos, garantizando que solo se construye lo que tiene impacto medible.

- **Specification by Example (SbE)**: definir los criterios de aceptación como ejemplos concretos en lugar de reglas abstractas, usando tablas o escenarios Gherkin [**Adzic2011**].

## 1.2 Mapa síntesis de la Semana 4

**Tabla 1.6.** *Mapa de cierre de la Semana 4.*

| Subtema | Concepto clave | Aplicación en el proyecto |
|---|---|---|
| 6.1 Prototipado | Throwaway vs. evolutivo; wireframe / mock-up. | Prototipo de baja fidelidad antes del taller. |
| 6.2 i* y KAOS | Actores, dependencias, jerarquía de metas. | Modelo i* SD del contexto organizacional. |
| 6.3 BPMN | Eventos de negocio → RF; lanes → autorización. | Diagrama AS-IS del proceso principal. |
| 6.4 Historias | INVEST; Gherkin Given/When/Then; épicas. | Backlog inicial si el proyecto es ágil. |

## 1.3 Práctica experimental y trabajo autónomo

**Práctica / Actividad**

**Práctica experimental n.º 4 (en clase, 2 horas).**

**Objetivo:** construir un modelo i* SD y un diagrama BPMN simple a partir de un caso dado.

**Caso:** un Gobierno Autónomo Descentralizado (GAD) municipal quiere automatizar el proceso de pago de impuesto predial, actualmente realizado en ventanilla con intervención manual de recaudadores, jefes de rentas y el sistema del Banco del Pacífico.

**Tareas en equipos de 3–4 estudiantes:**

1. Construir el modelo i* SD: identificar todos los actores (al menos 5), sus dependencias y sus tipos (meta, tarea, recurso, softgoal).
2. Construir el diagrama BPMN AS-IS del proceso actual en 3 lanes.
3. Derivar al menos 10 candidatos a RF y 3 a RNF a partir del diagrama BPMN.
4. Reescribir los 3 RF más importantes como historias de usuario con sus criterios de aceptación en formato Gherkin.

**Práctica / Actividad**

**Tarea autónoma n.º 4 (4 horas, en equipo del proyecto integrador).**

**Entregable para el hito 1 (continuación):**

1. **Modelo i* SD** del contexto organizacional del proyecto real.
2. **Diagrama BPMN AS-IS** del proceso principal que el sistema soportará (en la herramienta que el equipo elija: draw.io, Camunda, Lucidchart).
3. **Lista de eventos de negocio**: tabla con evento → RF derivado.
4. Si el proyecto es ágil: **story map** con épicas y las 5 primeras historias de usuario con criterios de aceptación Gherkin.

## 1.4 Resumen ejecutivo de la semana

- El **prototipado** convierte suposiciones implícitas en afirmaciones verificables; su tipo (desechable / evolutivo) debe decidirse antes de construirlo [**Floyd1984**, **Wiegers2013**].
- El framework **i*** modela las dependencias entre actores sociales y permite analizar el contexto organizacional antes de definir los requisitos del sistema [**Yu2011**].
- **KAOS** organiza los requisitos como una jerarquía de metas descompuestas con operadores AND/OR hasta llegar a requisitos operacionalizables [**vanLamsweerde2001**].
- Los diagramas **BPMN 2.0** permiten derivar sistemáticamente RF a partir de eventos, tareas y gateways del proceso de negocio [**Robertson2012**, **OMG_BPMN_2013**].
- Las **historias de usuario** con criterios de aceptación en formato **Gherkin** son el puente entre la elicitación ágil y las pruebas de aceptación automatizadas [**Cohn2004**, **Adzic2011**, **North2006**].
- El criterio **INVEST** garantiza la calidad de cada historia de usuario antes de incorporarla al backlog [**Cohn2004**].